

Order matters

Certified noncommutative consequences,
matrix countermodels, and operator inequalities

Algorithms, complete fragments, executable prototypes,
and a Lean integration plan

Prepared for Vladimir Reshetnikov
15 September 2026

Abstract

Forge already proposes induction planning, witness synthesis, commutative polynomial certificates, finite-dimensional and ideal closures, and specialized Ore-operator transport. This report adds a different mathematical worker: reasoning in a free associative algebra while preserving multiplication order. Its outputs are two-sided ideal receipts, finite rational matrix countermodels, and separate operator-positive and trace-positive certificates.

The central exact result is a finite decision procedure for consequences of homogeneous relations. A target of degree D needs only relation contexts of total degree at most D ; failure yields a concrete matrix model of a truncated quotient. A second exact fragment reduces homogeneous Hermitian-square search at arbitrary even degree to a unique rational Gram matrix. Mixed-degree and conditional positivity use bounded ray search modulo the equality space. A structural involution/commutation graph proposes the rays used in the CHSH worked example automatically.

The delivered Python implementation uses only the standard library. Its independent replay program accepts 119 stored evidence objects, including 45 matrix countermodels; 57 unit tests pass and 195 deliberately invalid mutations are rejected. These are object-language experiments, not Lean proofs or tactic benchmarks. The accompanying Lean reconstruction targets were not compiled.

Delivery boundary. No installed `forge` tactic, Lean reifier, verified computational checker, or measured gain against `grind` is claimed. The contribution is an implemented search/check layer with proved mathematical contracts and an explicit path to kernel-checked integration.

Contents

1	What is added, and what is deliberately not repeated	3
1.1	A revision-pinned boundary	3
1.2	The capability delta	3
1.3	A stronger baseline than bare normalization	4
1.4	Why this addition is useful	4
2	The object language and its exact interpretations	4
2.1	Words instead of exponent vectors	5
2.2	Four meanings, not four names for the same object	5
2.3	Involution and matrix semantics	6
3	Two-sided ideal receipts	6
3.1	The receipt and its soundness	6
3.2	A concrete bounded producer	7
3.3	Exact incidence slicing before elimination	7
3.4	Why an unsuccessful bounded search is not a refutation	8
4	A complete homogeneous fragment with matrix witnesses	8
4.1	Why degree D suffices	8
4.2	An executable construction, not an existence-only argument	9
4.3	What the completeness statement does and does not imply	10
4.4	The algorithmic consequence for Forge	10
5	Operator positivity through adjoint sandwiches	10
5.1	The correct replacement for scalar constraint products	10
5.2	A projection example that requires equality cooperation	11
6	Automatic homogeneous squares at arbitrary even degree	11
6.1	Why the noncommutative Gram matrix is unique	11
6.2	Relation to matrix positivity, without overstating the software	12
6.3	An automatic degree-six example	13
7	Mixed-degree search modulo equalities	13
7.1	Quotient the equality span before solving for positive weights	13
7.2	What the implemented support search guarantees	14
7.3	Automatic proposals from involution and commutation relations	14
8	A worked operator bound: the CHSH expression	15
8.1	The mathematical identity	15
8.2	What the producer actually discovers	15
8.3	The failed first run and its correction	15
9	Trace positivity and explicit cyclic remainders	16
9.1	A genuinely different certificate kind	16
9.2	A finite, constructive cyclic normal form	16
9.3	An exact separation between operator and trace conclusions	17
9.4	Why not turn cyclic equivalence into rewriting everywhere?	17
10	Integrating the worker into Lean and Forge	17
10.1	Reuse the host architecture, not its scalar representation	17
10.2	A proposed module split with narrow responsibilities	18
10.3	A proof-producing source bridge	19

10.4	Two reconstruction stages, with measurable gates	19
10.5	Concrete matrix library dependencies	20
10.6	Concrete optional search backends	20
10.7	A routing algorithm the planner can implement	21
10.8	Proof size and cache discipline specific to this worker	22
11	An incremental implementation plan with acceptance tests	22
11.1	Gate A: one real source-level equality	22
11.2	Gate B: reusable equality search	23
11.3	Gate C: operator and trace proof reconstruction	23
11.4	Gate D: exact-fragment diagnostics	23
11.5	Gate E: broader search and a fair comparative corpus	23
12	Executed experiments and their limits	23
12.1	Environment and independent replay	23
12.2	Three exact equality oracles	24
12.3	Forty homogeneous Gram queries	24
12.4	Nine worked obligations	25
12.5	An ablation with the same producer and target	25
12.6	Tests and mutation audit	26
12.7	What was not executed	26
13	Reproducing and extending the package	26
13.1	Files that matter	26
13.2	Reproduction commands	27
13.3	A direct Python use example	27
14	Conclusion	28
	References	28

1 What is added, and what is deliberately not repeated

1.1 A revision-pinned boundary

This report was prepared against Forge commit `c98e47c5f804e92880fc1d0e378c1b95832b685c`, not the earlier 6745210 design-round snapshot. The current README describes a 111-page merged article and eighteen contributing proposals, including an extension round. Its later merge records elaboration of sixteen core-only Lean files; that progress must not be erased by repeating the original proposals’ older “Lean unavailable” status. It still does not install a Forge tactic or report comparative tactic performance [1].

The review used the current README, merged provenance ledger, baseline section, scalar nonlinear section, the principal closure sections, and the Leant/Djex review. It also inspected the current Leant and Djex entry documents at the revisions in Table 1. This was a targeted source audit, not an exhaustive line-by-line review of every frozen submission. “Additional to Forge” below means additional to the mechanisms identified in that inspected merged baseline. It is not a claim of new mathematics or a priority claim over the noncommutative-algebra literature.

Project	Inspected revision
Forge	<code>c98e47c5f804e92880fc1d0e378c1b95832b685c</code>
Leant	<code>6bf05ad78c467989e68290f2d08bbbed40802d485</code>
Djex	<code>e8778f4ebd63e1f9b9b410fa4de8d14a8a04c9e5</code>

Table 1: Repository read snapshots. Mathlib documentation was consulted live; this is not a tested joint dependency lockfile.

1.2 The capability delta

Existing ingredient	Not a new contribution here	Actual addition
Commutative ideal certificates	Tracking polynomial multipliers and checking an expanded identity	Two-sided word contexts urv ; exact degree-complete homogeneous consequences and matrix witnesses.
Scalar quadratic positivity	Exact rational LDL^T as a numerical linear-algebra method	The unique word Gram matrix for homogeneous degree $2d$, with m^d words and no commutative coefficient collisions.
Scalar cone certificates	Nonnegative combinations of preselected squares	Adjoint sandwiches q^*gq , equality-space projection, and a structural ray generator for involutive operators.
Ore transport	Noncommutativity itself	General free-word relations, not a single prescribed shift/coefficient commutation law.

Existing ingredient	Not a new contribution here	Actual addition
Matrix observables and tensor lifts	Matrices as finite state-transition data	Symbolic matrix/operator inequalities valid independently of matrix dimension.
General certificate architecture	Opaque receipts, process separation, exact source identity, and status distinctions	Interpretation-specific replay: algebra equality, operator positivity, trace positivity, and algebra countermodel.

The previous scalar acceptance language uses commuting real indeterminates and products of nonnegative constraints; the existing closure and Ore mechanisms have their own explicit semantic fragments [2, 3]. Neither interface should be silently widened to the new one. In particular, positive semidefinite matrices cannot be multiplied as though they were nonnegative real numbers.

1.3 A stronger baseline than bare normalization

Mathlib already has `noncomm_ring`. It expands noncommutative ring expressions using simplification and additive normalization, accepts supplied rewrite lemmas and local hypotheses, and documents limitations around fully expanded powers [8]. Therefore this proposal is not “add a noncommutative normalizer.” Its search constructs *new combinations of hypotheses in left and right contexts*, and its other outputs include countermodels and positivity receipts. Some worked equalities may already be easy for `noncomm_ring [*]`, `simp`, or `grind`. They are replay and integration examples, not evidence that existing tactics fail.

The current `grind` remains a cooperating collection of proof-producing engines, not a trivial baseline [7]. A fair later evaluation needs `grind`, `noncomm_ring`, `noncomm_ring [*]`, well-chosen `simp` rules, and the same workers without information sharing. None of those comparisons was executed here.

1.4 Why this addition is useful

Matrices, endomorphisms, projections, and operator expressions produce the same obstruction: one knows equations about products, but the useful occurrence of a relation is buried inside a larger product. For instance,

$$UA = 1, \quad BV = 1 \implies U - V = U(B - A)V.$$

An equality worker should find the two surrounding contexts instead of asking a human to insert them. Likewise, $G \succeq 0$ naturally proves $Q^*GQ \succeq 0$, not $GQ \succeq 0$. And $\text{tr}([A, B]) = 0$ is a scalar trace fact, not the assertion $[A, B] = 0$.

These are useful new outputs for a planner: an equality for local saturation, a proved positive operator for an analytic subgoal, a scalar trace inequality, or a concrete diagnostic against an over-general algebraic conjecture. They need not replace any existing Forge worker.

2 The object language and its exact interpretations

2.1 Words instead of exponent vectors

Let $X = \{x_0, \dots, x_{m-1}\}$ and let X^* be its finite words, including the empty word ϵ . A polynomial in the free associative unital algebra

$$\mathcal{F} = \mathbb{Q}\langle x_0, \dots, x_{m-1} \rangle$$

is a finitely supported map $X^* \rightarrow \mathbb{Q}$. Multiplication concatenates words; xy and yx remain distinct. The empty word represents 1.

The implementation stores a word as a tuple of atom indices and a polynomial as an exact sparse dictionary. Serialization uses sorted triples `[word, numerator, denominator]`, with nonzero reduced coefficients and positive denominators. The checker implements its own accumulation and product routines; it does not import the producer's polynomial arithmetic or elimination code. The representation and wire schema remain shared design choices, so this is algorithmic separation, not protection against every common-mode mistake.

Rational coefficients are *central*. A Lean source translation must target an associative $\mathbb{Q}$ -algebra, not an arbitrary ring. The implication $2x = 0 \Rightarrow x = 0$ admits the rational receipt $x = \frac{1}{2}(2x)$ but is false in characteristic two. Merely changing a pretty-printed scalar type would invalidate the interface. Integer-coefficient receipts can be reconstructed more generally, as several of the supplied Lean targets illustrate.

2.2 Four meanings, not four names for the same object

The problem and certificate schemas distinguish the following cases.

Algebra equality.

For every associative unital $\mathbb{Q}$ -algebra and every tuple satisfying the listed relations $r_j = 0$, the target p is zero. The prototype schema intentionally has no additional domain restrictions.

Algebra countermodel.

Concrete rational square matrices satisfy every listed relation as a whole matrix, and a supplied vector is separated by p . This disproves the preceding universal algebra equality. It does not refute a goal about a particular matrix size, self-adjoint matrices, or a restricted algebra unless those extra conditions are separately checked.

Operator positivity.

For real square matrices of arbitrary finite dimension, satisfying the declared transpose involution on atoms, the relations, and the positive semidefinite premises $g_i \succeq 0$, the target satisfies $p \succeq 0$.

Trace positivity.

Under the same matrix assumptions, $\text{tr}(p) \geq 0$. This is strictly weaker than operator positivity and accepts a different residual language.

The checker requires the certificate kind to agree with the original problem, except for the explicitly permitted equality/countermodel alternative. Unknown fields such as an unsupported fixed matrix dimension or finite-field tag are rejected rather than ignored. This does not solve the Lean source-translation problem: a future reifier must first prove that the source goal has the interpretation being requested.

2.3 Involution and matrix semantics

An involution on atom indices is a permutation $i \mapsto i^*$ with $(i^*)^* = i$. Extend it by

$$(x_{i_1} \cdots x_{i_k})^* = x_{i_k^*} \cdots x_{i_1^*}, \quad c^* = c \quad (c \in \mathbb{Q}).$$

Fixed atoms represent self-adjoint matrices. A two-cycle can represent a matrix and its transpose. Both the reversal and the index map are indispensable. The executed positivity corpus mostly uses fixed atoms; a separate unit test covers paired atoms. The finite-ray producer currently assumes fixed atoms, whereas the homogeneous Gram producer accepts either kind.

The operator and trace schemas require a formally self-adjoint target and formally self-adjoint positive premises. Self-adjointness that follows only *modulo relations* requires an extra equality obligation or a rewritten self-adjoint expression. This conservative entrance restriction caused a real failed experiment in Section 8; it was preserved, not bypassed.

Complex matrices and abstract ordered star algebras are natural extensions of the same mathematical proof rules. The delivered schema fixes real matrix semantics; a larger interpretation deserves its own proved adapter rather than an undocumented change of scope.

3 Two-sided ideal receipts

Evidence status. Producer, independent Python replay, exact oracle comparisons, and adversarial tests are implemented. Lean source reification is not implemented.

3.1 The receipt and its soundness

For supplied relations $r_1, \dots, r_s$ and target p , a receipt consists of a finite list of rational coefficients, relation indices, and two words:

$$p = \sum_{\ell=1}^t c_\ell u_\ell r_{j_\ell} v_\ell. \tag{1}$$

The left and right words are separately represented fields. The original relations and target are arguments to replay, not assertions made by the receipt.

Theorem 3.1 (Two-sided receipt soundness). *If identity (1) holds in $\mathcal{F}$, then under every evaluation into an associative unital $\mathbb{Q}$ -algebra satisfying $r_j = 0$, the evaluated target is zero.*

Proof. Evaluation is an algebra homomorphism, so it preserves the finite sum and each product in (1). Every summand has the factor $r_{j_\ell} = 0$. No commutation, division by an operator, or cancellation of a nonzero element is used. Rational coefficients act through the central scalar map. $\square$

A receipt need not mention a Gröbner basis, a monomial ordering, or a minimal derivation. Those are possible discovery mechanisms, not facts needed for this conclusion. Compared with the existing commutative multiplier format, the essential change is not replacing an implementation name: it is preserving both sides of each relation occurrence.

3.2 A concrete bounded producer

Choose a degree bound $D \geq \deg p$. For each relation of degree d_j , enumerate all pairs of words (u, v) with $|u| + |v| \leq D - d_j$ and expand the column ur_jv . A sparse exact elimination basis tracks each admitted row as a rational combination of these original columns. Reducing p against that basis either returns the coefficients in (1) or a nonzero remainder.

```
columns := [(j,u,v, expand(u * relation[j] * v))
             for j,u,v with len(u)+degree(relation[j])+len(v) <= D]
selected := incidenceComponent(target.support, columns)
basis := empty sparse exact echelon basis with provenance
for column in selected:
    insert(column, original_column_index)
(coefficients, remainder) := reduce(target, basis)
if remainder != 0:
    return UNKNOWN_OUTSIDE_BOUNDED_CONTEXT_SPAN
receipt := original-context terms selected by coefficients
return receipt only after independent expansion succeeds
```

The prototype performs Gaussian elimination over `fractions.Fraction`. A production backend can replace it by sparse fraction-free or modular exact linear algebra without changing the receipt format. The relevant finite feasibility problem is linear; there are no unknown generators being multiplied by unknown multipliers.

Complexity. There are at most

$$R_D = \sum_{k=0}^D m^k \quad \text{word rows,} \quad C_D = \sum_{j=1}^s \sum_{\ell=0}^{D-d_j} (\ell+1)m^\ell \quad \text{context columns.} \quad (2)$$

An upper sum with $D - d_j < 0$ is empty. The factor $\ell + 1$ counts the possible split between the two contexts of a length- ℓ word. Dense elimination costs $O(R_D C_D \min(R_D, C_D))$ field operations; sparse behavior and rational bit growth determine the practical cost. These counts are exponential in D , not a claim of a generally efficient noncommutative theorem prover. The implemented column cap is 20,000; exceeding it raises an explicit budget exception that a controller must map to `UNKNOWN`.

3.3 Exact incidence slicing before elimination

Construct a bipartite graph with word rows on one side and context columns on the other; an edge means the coefficient of that word in that column is nonzero. Keep every connected component that meets the target's support. This is more precise than merely keeping all columns mentioning one of the target's variables.

Proposition 3.2 (Component slicing is exact). *The target lies in the span of all columns if and only if it lies in the span of the retained columns.*

Proof. One implication is immediate. For the other, partition a hypothetical solution into graph components. Distinct components have disjoint row supports. Every component disjoint from the target must sum to zero on its own rows, independently of the components meeting the target. Removing all such contributions leaves a solution for the target using only retained columns. $\square$

No heuristic premise deletion is hidden in this argument. Conversely, slicing only one hop from the target would be wrong: a column may introduce another word whose cancellation requires a further column. The prototype uses a worklist until the whole incidence component is exhausted.

The executed implementation assembles *all* columns before constructing the graph. Its measured benefit is reduced elimination, not avoidance of the exponential column-assembly cost. A future lazy context index may improve that cost, but it needs a completeness argument for enumerating every incident column. The current result must not be described as such a lazy algorithm.

3.4 Why an unsuccessful bounded search is not a refutation

Set

$$r_U = U(1 - AB) - 1, \quad r_V = (1 - BA)V - 1, \quad p = UA - AV.$$

The following exact identity holds in the free algebra:

$$p = r_U AV - U A r_V. \quad (3)$$

The target has degree two, but the displayed context products have degree five. In the delivered experiment, context bounds two, three, and four all return UNKNOWN, whereas bound five returns a receipt. This is a direct operational control against equating “target degree” with a sufficient proof degree for inhomogeneous presentations.

There is a sharper semantic warning. The relation $xy = 1$ does not imply $yx = 1$ in every algebra: left and right shifts on the countably generated vector space provide a counterexample. If $Se_i = e_{i+1}$ and $Le_0 = 0$, $Le_{i+1} = e_i$, then $LS = 1$ but $SL \neq 1$. Yet square matrices over a field with $XY = I$ also satisfy $YX = I$, since a right inverse in finite dimension is an inverse. Thus a finite-matrix search is not complete for arbitrary inhomogeneous algebra counterexamples. The homogeneous restriction in the next section is doing substantial work.

4 A complete homogeneous fragment with matrix witnesses

Let $\mathcal{F}_k$ be the rational span of words of length k . A relation is homogeneous when all its nonzero monomials have the same degree. Different relations may have different degrees; zero relations are harmless. Let I be their two-sided ideal and set

$$\mathcal{F}_{\leq D} = \bigoplus_{k=0}^D \mathcal{F}_k, \quad \mathcal{F}_{> D} = \bigoplus_{k>D} \mathcal{F}_k.$$

The second space is a two-sided ideal: multiplying a word of length greater than D by any word cannot shorten it.

4.1 Why degree D suffices

Lemma 4.1 (Degree-bounded membership). *For homogeneous relations and $p \in \mathcal{F}_{\leq D}$, membership $p \in I$ is equivalent to membership in the finite span*

$$\text{span}_{\mathbb{Q}}\{ur_jv : |u| + d_j + |v| \leq D\}.$$

Proof. The reverse implication follows from the definition of an ideal. For the forward implication, write a finite ideal representation of p and expand both polynomial multipliers into words. Each resulting context product is homogeneous. Group these products by total degree. Every group of degree greater than D sums to zero, since p has no component in that degree. Delete those groups. The remaining products satisfy the asserted degree bound. $\square$

This proof does not appeal to termination of noncommutative completion or to a Noetherian property. It isolates a finite-dimensional vector-space problem at the exact required degree. It needs no global completion result for the presented algebra.

Theorem 4.2 (Exact graded alternative). *Given finitely many homogeneous rational relations and a target of degree at most D , exact elimination with all contexts of total degree at most D returns either a receipt (1), or rational square matrices $X_0, \dots, X_{m-1}$ and a rational vector v such that*

$$r_j(X_0, \dots, X_{m-1}) = 0 \quad \text{for every } j, \quad p(X_0, \dots, X_{m-1})v \neq 0.$$

A witness dimension no larger than R_D in (2) suffices.

Proof. If elimination succeeds, Theorem 3.1 applies. Otherwise Lemma 4.1 gives $p \notin I$. Because I is graded,

$$p \notin I + \mathcal{F}_{>D}.$$

Indeed, the degree-at-most- D part of any element of I is again in I . Consider the finite-dimensional quotient algebra

$$A_D = \mathcal{F}/(I + \mathcal{F}_{>D}).$$

Its dimension is at most R_D . The image of p is nonzero, so this quotient is not the zero algebra and its identity is nonzero. Let X_i be the matrix of left multiplication by the image of x_i in any rational basis of A_D , and let v represent its identity. Left multiplication is a unital algebra homomorphism. Consequently all input relations evaluate to the zero matrix, whereas $p(X)v$ is exactly the nonzero coordinate vector of the image of p . $\square$

4.2 An executable construction, not an existence-only argument

The producer forms the full degree-bounded echelon basis. Nonpivot words form a basis of A_D . To construct column j of X_i , prepend x_i to the j th basis word and reduce it; a word longer than D reduces to zero by truncation. Reduce the empty word to construct v .

```

E := exact basis of all relation contexts through degree D
B := all nonpivot words of length at most D
for atom i:
  for basis word B[j]:
    X[i][:,j] := coordinates(normal_form(i :: B[j]))
v := coordinates(normal_form(empty_word))
return X,v only after matrix replay on the ORIGINAL relations

```

Unlike the equality receipt, this producer must use the *full* relation space, not just the target's incidence component. A sliced equality proof is sound; a quotient that omits unrelated relations need not model the whole presentation. The implementation uses full elimination here deliberately. The checker does not reconstruct a quotient, inspect pivots, verify a basis, or trust the producer's homogeneity assertion. It multiplies the supplied rational matrices and verifies every complete

relation matrix, then the separating target vector. An accidentally inapplicable construction cannot pass merely because it looks like a quotient. The producer rejects inhomogeneous input before attempting this construction; the matrix checker itself can accept a directly valid countermodel for any equality-schema relation list.

A crucial independent test. Take the purported relation $X = 0$, the matrix $X = \text{diag}(0, 1)$, target 1, and vector e_1 . Then $Xe_1 = 0$ and $1e_1 \neq 0$, but $X \neq 0$. Checking the relation only on the witness vector would accept a false countermodel. The supplied unit suite rejects exactly this object.

4.3 What the completeness statement does and does not imply

For homogeneous presentations, the theorem decides the universal algebra consequence and, equivalently, the consequence over rational matrices of all finite sizes. It does not decide the corresponding consequence in one selected algebra, nor with omitted positivity, involution, or invertibility assumptions. The quotient matrices need not be symmetric, and their dimension is whatever the quotient needs. They are not automatic counterexamples to a goal about 2×2 Hermitian matrices.

Operational caps are a separate boundary. The implemented producer refuses an ambient word count above 4,000 or a quotient dimension above 100. Such a refusal is `UNKNOWN` under a resource limit, even though the uncapped mathematical procedure terminates. No matrix witness is promised below an arbitrarily chosen user cap.

4.4 The algorithmic consequence for Forge

This worker has an unusually clean escalation policy. Homogeneous equalities need only one degree level, followed by either identity replay or matrix replay. Inhomogeneous equalities use increasing context bounds and never turn a nonzero bounded remainder into a source-level refutation. This distinction is available syntactically before the expensive search begins. It is more useful than a generic instruction to “try a bigger algebra solver.”

5 Operator positivity through adjoint sandwiches

Evidence status. Independent operator replay, an automatic homogeneous Gram producer, and a bounded finite-ray producer are implemented. General SDP discovery and Lean positivity reconstruction are integration work, not executed features.

5.1 The correct replacement for scalar constraint products

For real symmetric positive semidefinite G and any real matrix Q ,

$$z^T(Q^T G Q)z = (Qz)^T G (Qz) \geq 0.$$

Thus congruence preserves positivity. In contrast, let

$$A = \begin{pmatrix} 1 & 0 \\ 0 & 0 \end{pmatrix}, \quad B = \begin{pmatrix} 1 & 1 \\ 1 & 1 \end{pmatrix}.$$

Both are positive semidefinite, but AB is not symmetric. Even the symmetric product $AB + BA$ is not positive semidefinite: at $z = (1, -2)^\top$ its quadratic form is -2 . This exact arithmetic example is an executed regression test, not just a caution in the prose.

The operator receipt therefore has the form

$$p = \sum_{k=1}^t d_k q_k^* g_{i_k} q_k + \sum_{\ell=1}^u c_\ell a_\ell r_{j_\ell} b_\ell, \quad d_k \in \mathbb{Q}_{\geq 0}, \quad g_0 = 1. \quad (4)$$

The original problem supplies $g_i \succeq 0$ and $r_j = 0$. The equality terms have no sign restrictions and preserve their left/right word contexts. Here the q_k are polynomials, whereas a_ℓ, b_ℓ are words; polynomial equality multipliers can be expanded into finitely many such word contexts.

Theorem 5.1 (Operator receipt soundness). *A checked identity (4) implies $p(X) \succeq 0$ for every matrix tuple satisfying the original involution, equality, and positivity assumptions.*

Proof. The equality contribution evaluates to zero by the same argument as Theorem 3.1. Each remaining summand is positive semidefinite by congruence and nonnegative scalar multiplication. Their sum is positive semidefinite. The involution contract makes the formal q_k^* evaluate to the transpose of the evaluated q_k . $\square$

The proof does not assert that this certificate language expresses every valid conditional matrix inequality. In particular, a finite dictionary of q_k is a further restriction. Failure of that search is not a mathematical counterexample.

5.2 A projection example that requires equality cooperation

Suppose $x^* = x$ and $x^2 = x$. Then

$$x = x^*x - (x^2 - x)$$

is a one-square operator receipt. The target has degree one, so it is not in the free homogeneous-square fragment below. The equalities are what make it positive. The implemented finite-ray worker finds the weight and the ideal residual rather than trying to treat the premise $x^2 - x = 0$ as a positivity constraint.

Similarly, from $x \succeq 0$ the target xyx with self-adjoint y is certified by the sandwich y^*xy . Deleting the positive premise makes the exact same receipt invalid. Both cases are covered by the stored worked examples and boundary tests.

6 Automatic homogeneous squares at arbitrary even degree

6.1 Why the noncommutative Gram matrix is unique

Fix $d \geq 0$ and list all words of length d as $W_d = (w_1, \dots, w_N)^\top$, where $N = m^d$. A homogeneous polynomial p of degree $2d$ has a unique expansion

$$p = W_d^* Q W_d = \sum_{i,j} Q_{ij} w_i^* w_j. \quad (5)$$

Indeed, every word of length $2d$ has a unique split into its first and last d letters; the first half uniquely determines w_i by reversing and applying the atom involution. There are $m^{2d} = N^2$ words

and N^2 ordered matrix entries. Formal self-adjointness of p is exactly symmetry of the rational matrix Q .

This is a decisive difference from ordinary commuting multivariate polynomials, where different products of monomials can have the same exponent vector and Gram representations need not be unique. The existing scalar quadratic *LDL* mechanism does not, by itself, expose this arbitrary-even-degree free-word fragment. The new step is the word splitting and its exact coefficient correspondence, not the invention of *LDL* factorization.

Theorem 6.1 (Complete homogeneous-square fragment). *For a rational homogeneous formally self-adjoint p of degree $2d$, the following are equivalent:*

- (i) *Its unique word Gram matrix Q is positive semidefinite.*
- (ii) *$p = \sum_k d_k q_k^* q_k$, with rational $d_k \geq 0$ and rational homogeneous q_k of degree d .*
- (iii) *p has a sum-of-Hermitian-squares representation using real homogeneous polynomials of degree d .*

The rational representation can be found by exact pivoted $LDL^\top$ elimination, without numerical semidefinite optimization.

Proof. A representation in (iii) expresses Q as a sum of real outer products by uniqueness in (5), proving (i). Condition (ii) immediately implies (iii), absorbing $\sqrt{d_k}$ into the real polynomials.

For (i) to (ii), work entirely over $\mathbb{Q}$. If a diagonal entry $Q_{kk} = a$ is positive, complete the square using the row

$$\ell = e_k + \sum_{j \neq k} \frac{Q_{kj}}{a} e_j.$$

Subtract $a\ell\ell^\top$ and recurse on the remaining Schur complement. It is positive semidefinite and rational. If all remaining diagonal entries are zero, positive semidefiniteness forces every off-diagonal entry to be zero: a nonzero Q_{ij} would give a negative quadratic form on a suitable vector supported on i, j . Thus the process terminates with $Q = \sum_k d_k \ell_k \ell_k^\top$, $d_k \in \mathbb{Q}_{>0}$. Set $q_k = \sum_i (\ell_k)_i w_i$ and use (5). $\square$

The zero polynomial is accepted with an empty square list. A nonzero polynomial of odd homogeneous degree is outside this entrance fragment. The matrix test is exact: an off-diagonal nonzero entry with zero diagonal is a rejection, not an invitation to add a numerical tolerance.

6.2 Relation to matrix positivity, without overstating the software

Helton's theorem identifies matrix-positive free polynomials in self-adjoint variables with sums of Hermitian squares [11]. It supplies relevant mathematical context; the present report does not claim that theorem as new. For a homogeneous target, square factors can be taken homogeneous of half the degree. To see the degree restriction, consider the highest homogeneous pieces of putative factors: their sum of squares cannot cancel, because the unique word Gram matrix is a sum of outer products and is nonzero if any piece is nonzero. Apply the same argument to the lowest nonzero pieces. A pure degree- $2d$ sum therefore has no factor degrees above or below d .

Consequently, for self-adjoint free variables and all finite matrix dimensions, the exact Gram criterion also characterizes homogeneous matrix positivity. That mathematical corollary does *not* mean the delivered negative output is a Lean proof of nonpositivity. The routine returns no certificate when Q is not positive semidefinite. It does not construct a self-adjoint matrix tuple violating the inequality,

and the checker never accepts a Gram failure as such a countermodel. The separately implemented quotient-matrix construction is an *equality* countermodel mechanism with a different scope.

6.3 An automatic degree-six example

Let self-adjoint x, y be free and set

$$q = x^2y + 2yx^2 - 3y^3, \quad p = q^*q.$$

The experiment supplies only the expanded target p to `homogeneous_sos`. There is no supplied factor or dictionary. The producer builds an 8×8 Gram matrix, factors it rationally, and emits a square receipt which independent replay expands back to p . The recovered factor need not have the same normalization as the generating q ; the nonnegative rational weight accounts for that harmless freedom.

The cost is $O(m^{3d})$ dense rational field operations in the worst case, with an m^{2d} -entry Gram matrix. Sparse word support and block decomposition may reduce that cost; a six-variable degree-twelve goal is not made small merely by being in a decidable fragment. A production implementation needs explicit word, coefficient-bit, and factor-size budgets.

7 Mixed-degree search modulo equalities

7.1 Quotient the equality span before solving for positive weights

Fix a context bound D , form the equality column span M_D from Section 3, and choose a finite family of candidate rays

$$a_k = q_k^* g_{i_k} q_k.$$

Let π_D be the linear remainder map of the exact echelon basis. Search for

$$\pi_D(p) = \sum_k d_k \pi_D(a_k), \quad d_k \geq 0. \tag{6}$$

After obtaining the weights, reduce the *original* residual $p - \sum_k d_k a_k$ to recover its two-sided ideal receipt. This avoids presenting all equality coefficients as free signed variables in the positive feasibility problem.

Proposition 7.1 (Equality projection loses no fixed-language solution). *For the fixed equality span and fixed rays, equation (6) is feasible if and only if there exists a receipt (4) using those rays and equality contexts in M_D .*

Proof. The kernel of the linear remainder map is precisely M_D . Thus the projected identity holds exactly when the difference of target and weighted rays belongs to M_D , which is exactly what the equality terms express. $\square$

It would be incorrect to run positivity directly on the possibly non-self-adjoint-looking remainder and forget the target. A normal-form map is only a linear coordinate device; the final replay always checks the original self-adjoint target, the original positive premises, and the original relations.

7.2 What the implemented support search guarantees

The standard-library producer enumerates supports of size one through a given cap, solves the corresponding exact linear systems, and accepts a representation only if every recovered coefficient is nonnegative. It defaults to support size at most four and at most 10,000 attempted supports. It is not an SDP solver.

There is a precise completeness statement for an uncapped finite-ray version. If a vector lies in the cone generated by finitely many vectors in a rational space of dimension r , it has a representation on a linearly independent support of size at most r . Starting with a nonnegative representation, a linear dependence of its positive-support vectors allows movement along a coefficient null vector until a positive coefficient first becomes zero, without making any coefficient negative. Repeat until the support is independent. An independent rational support has unique rational coefficients for a rational target. Hence exhaustive support enumeration through r finds a solution when one exists. With a smaller support or attempt cap, failure remains UNKNOWN.

Dependent supports are not a loophole in this argument: their feasible representations can be reduced to an independent support, which is eventually visited by the uncapped enumerator. The result is completeness for this *fixed cone*, not for conditional noncommutative positivity.

7.3 Automatic proposals from involution and commutation relations

The delivered function `involution_dictionary` adds a concrete source of rays rather than leaving every q_k to a user. It recognizes nonzero scalar multiples of

$$x_i^2 - 1, \quad x_i x_j - x_j x_i$$

in the supplied relation list. These identify involutive atoms and explicitly known commuting pairs. For two involutive atoms not explicitly marked commuting, it proposes both their commutator and their anticommutator. For two disjoint pairs whose cross-pairs are explicitly commuting, it also proposes the sum and difference of their commutators.

```
U := atoms with a recognized relation x_i*x_i - 1 = 0
E := pairs with a recognized relation x_i*x_j - x_j*x_i = 0
P := pairs in U not already in E
propose x_i*x_j + x_j*x_i and x_i*x_j - x_j*x_i for each pair in P
for disjoint pairs p,q with every cross-pair in E:
  propose comm(p) + comm(q) and comm(p) - comm(q)
deduplicate nonzero scalar multiples in the SEARCH dictionary
send every resulting square proposal through exact weight and identity replay
```

Absence of a commutation relation is not used as a proof of noncommutation. It only makes a pair eligible for proposals. The dictionary does not learn an unproved relation, and a missed syntactic pattern only reduces search power. Indirectly derivable involution or commutation facts would have to be supplied as previously proved lemmas to this entrance. There is no completeness claim for the ray generator.

At most P eligible pairs give $2P + 2\binom{P}{2}$ candidates before filtering and deduplication. The current pair cap is twenty. The algorithm is small enough to port directly into a Lean-side proposal worker; its correctness need not be trusted because every eventual receipt is independently replayed.

8 A worked operator bound: the CHSH expression

8.1 The mathematical identity

Let A, D, B, C be self-adjoint involutions and assume every member of $\{A, D\}$ commutes with every member of $\{B, C\}$. Write

$$S = A(B + C) + D(B - C), \quad P = AD - DA, \quad Q = BC - CB,$$

$$T_A = AD + DA, \quad T_B = BC + CB.$$

The cross-commutations imply $S^* = S$ and $PQ = QP$. Expanding with the involution relations gives

$$S^2 = 4I - PQ, \quad T_A^2 - P^2 = 4I, \quad T_B^2 - Q^2 = 4I.$$

Since $P^* = -P$ and $Q^* = -Q$,

$$\frac{1}{2}(P - Q)^*(P - Q) + \frac{1}{2}T_A^*T_A + \frac{1}{2}T_B^*T_B = \frac{1}{2}(-P^2 + 2PQ - Q^2) + \frac{1}{2}T_A^2 + \frac{1}{2}T_B^2 \quad (7)$$

$$= 4I + PQ = 8I - S^2. \quad (8)$$

This proves $S^2 \preceq 8I$, and hence $\|S\| \leq 2\sqrt{2}$ in the usual finite-dimensional Euclidean operator norm. The norm implication follows by applying $S^*S \preceq 8I$ to each vector and taking the supremum over unit vectors. The recorded artifact certifies the polynomial inequality and a separate self-adjointness identity; it contains no Lean norm theorem.

This is a worked application of known operator mathematics, not a new physical bound. Its value here is that a noncommutative receipt can simultaneously use operator involution, selectively commuting groups, and a positive-square identity without pretending all four operators commute.

8.2 What the producer actually discovers

The source problem supplies four involution relations and four cross-commutation relations. From those relations the graph rule of Section 7.3 generates six candidate polynomials. The finite-ray worker searches their nonnegative combinations modulo degree-four equality contexts. It finds a three-square receipt with rational weights, then recovers a two-sided ideal residual. Neither the weights nor the list of three chosen factors is supplied to that search. This is a bounded structurally guided search, not a general discovery procedure for quantum inequalities.

The unit suite repeats the same construction under all twenty-four permutations of the atom names. Those are *metamorphic checks of one family*, not twenty-four independent mathematical discoveries or a twenty-four-goal tactic benchmark. They test whether pattern recognition or reconstruction accidentally depends on the particular ordering A, D, B, C .

8.3 The failed first run and its correction

The first authoring run used the target $8I - S^2$. As a *free* word polynomial, that expression is not syntactically self-adjoint: S becomes self-adjoint only through the cross-commutation hypotheses. The conservative operator producer correctly refused the input. The initial failing log is retained as `results/initial-failure.log`.

The repair was not to turn off the self-adjointness check. The executed operator target is

$$8I - S^*S,$$

which is formally self-adjoint. A separate exact equality receipt proves $S - S^* = 0$ using the cross-commutation relations. Substitution then gives the stated squared bound. This is an example of a useful Forge obligation exchange: an algebra worker proves the missing structural fact while the positivity worker retains its simpler, safer input language.

For this particular norm bound there is also a simpler discovery opportunity: if $U = A(B + C)$ and $V = D(B - C)$, then $U^*U + V^*V = 4I$ already follows from the involution relations, and the parallelogram identity gives $8I - (U + V)^*(U + V) = (U - V)^*(U - V)$. The graph-based three-square certificate is not minimal. A source-expression-aware sign-flip proposal could find the one-square version, but that alternative proposer was not implemented or counted in the recorded run. Retaining the distinction prevents a successful demonstration from being mislabeled a proof-size optimum.

9 Trace positivity and explicit cyclic remainders

9.1 A genuinely different certificate kind

For square matrices, $\text{tr}(AB - BA) = 0$. Consequently trace positivity permits

$$p = \sum_k d_k q_k^* g_{i_k} q_k + \sum_\ell c_\ell a_\ell r_{j_\ell} b_\ell + \sum_t e_t (u_t v_t - v_t u_t), \quad d_k \geq 0. \quad (9)$$

The final sum need not vanish as an operator. Its coefficients can have either sign. The checker expands every commutator exactly; it does not trust a flag saying that a residual is “cyclically zero.”

Theorem 9.1 (Trace receipt soundness). *A checked identity (9), under the original matrix assumptions, implies $\text{tr}(p) \geq 0$.*

Proof. The equality terms vanish, the commutator traces vanish by cyclicity, and the trace of each positive semidefinite sandwich is nonnegative. For real matrices the last claim follows directly by summing its nonnegative diagonal quadratic forms against the standard basis. $\square$

9.2 A finite, constructive cyclic normal form

Associate each word with the lexicographically least of its cyclic rotations. The cyclic normal form of a polynomial collects coefficients at these canonical words. If this normal form is zero, then

$$p = \sum_w c_w (w - \text{cyc}(w)),$$

since the coefficients of each rotation class sum to zero. Choose a split $w = uv$ whose rotation vu is canonical; its contribution is the explicit commutator $c_w(uv - vu)$. This proves that cyclic reduction to zero can be converted into the final receipt language without relying on a normalizer’s unexplained verdict.

The implemented search in trace mode uses this linear cyclic map in place of the equality remainder map. It currently accepts *no equality relations in its trace-search input*. The independent trace

checker can replay equality terms if a future producer supplies them, but that larger search was not implemented. This producer/checker asymmetry is intentional and recorded.

The elementary implementation enumerates rotations and costs quadratic time in a word’s length. More sophisticated minimal-rotation and necklace indexing can be an optimization later; it would not alter the explicit commutator receipt.

9.3 An exact separation between operator and trace conclusions

For self-adjoint x, y , let

$$h = [x, [x, y]] = x^2y - 2xyx + yx^2, \quad p = I + 2h.$$

The polynomial h is self-adjoint and has zero trace, so $\text{tr}(p) = n \geq 0$ for $n \times n$ matrices. The trace producer certifies it using the single square 1×1 and explicit cyclic residuals.

Take

$$x = \begin{pmatrix} 1 & 0 \\ 0 & 0 \end{pmatrix}, \quad y = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}.$$

Then $h = y$ and

$$p = \begin{pmatrix} 1 & 2 \\ 2 & 1 \end{pmatrix}, \quad \text{tr}(p) = 2, \quad (1, -1)p(1, -1)^\top = -2.$$

It is not positive semidefinite. Changing the receipt kind from trace to operator must not convert this successful trace proof into a matrix inequality. The strict kind/field checks and an exact quadratic-form test enforce this boundary in the delivered suite.

9.4 Why not turn cyclic equivalence into rewriting everywhere?

Cyclic rotation is legitimate under a trace, but not under arbitrary matrix contexts. From $\text{tr}(AB) = \text{tr}(BA)$ one cannot conclude $Q^*ABQ = Q^*BAQ$, or cancel a trace wrapper and return an equality to `grind`. Therefore the trace worker exports only a scalar trace theorem. A future typed frontend should preserve the observation wrapper as part of the obligation identity, rather than treating it as irrelevant syntax.

10 Integrating the worker into Lean and Forge

Evidence status. The module layout, source bridge, routing rules, and acceptance gates below are proposed. Only the Python components listed in Section 12 were executed. The six small Lean reconstruction targets are explicitly uncompiled.

10.1 Reuse the host architecture, not its scalar representation

Forge’s existing review of Leant and Djex already covers exact source ownership, status distinctions, opaque receipts, and the separation of discovery from verification [4]. Repeating that architecture as a contribution would add little. Here its important consequence is concrete: a noncommutative atom must retain its actual multiplication instance, scalar action, adjoint, and matrix index type.

Leant’s documented candidates retain source-level selections and are checked again by Lean; Djex distinguishes complete propositional search from bounded higher-rank synthesis [5, 6]. The useful transfer is to treat an operator expression proposed by those systems as a typed *candidate*, not as an algebraic theorem. Its semantics must still be established by the new receipt or an existing proof. Neither project was built or invoked in these experiments, and their historical test counts are not added to this report’s numbers.

The implementation should begin as an isolated `Forge.NC` worker at the existing tactic/obligation boundary. It need not reach into `grind`’s internal solver interfaces to test whether its new certificates help. Once a reconstructed equality is available, return an ordinary Lean theorem to the host’s existing assertion mechanism. Operator and trace results should likewise return the exact original proposition, not an untyped Boolean success flag.

10.2 A proposed module split with narrow responsibilities

Module	Required responsibility
<code>Forge.NC.WordPoly</code>	Canonical finite rational word polynomials; addition, concatenation, star, and evaluation theorems. No search state.
<code>Forge.NC.Reify</code>	Extract a supported expression and a proof that evaluating the object polynomial gives the original Lean expression. Preserve atom types and operation instances.
<code>Forge.NC.Receipt</code>	Typed equality, operator, trace, and matrix-witness formats. Source problem is a separate argument.
<code>Forge.NC.Check</code>	Executable identity checks and their soundness theorems. Use theorem-producing normalization first; reflection can replace it later.
<code>Forge.NC.Graded</code>	Optional formal account of homogeneous degree completeness and finite quotient witnesses. Not required for accepting a positive equality receipt.
<code>Forge.NC.Matrix</code>	Matrix evaluation, adjoint correspondence, congruence positivity, and trace-specific reconstruction.
<code>Forge.NC.Search</code>	Untrusted context enumeration, slicing, Gram factorization, ray proposal, and external backend adapters.
<code>Forge.NC.Worker</code>	Route supported obligations, enforce resource limits, and return either reconstructed proofs or appropriately scoped diagnostics.

A practical entrance vocabulary is:

```
needEquality(sourceGoal, sourceRelations, algebraProfile)
needOperatorPositive(sourceGoal, sourceRelations, positiveFacts, starProfile)
needTracePositive(sourceGoal, sourceRelations, positiveFacts, traceProfile)
needAlgebraCountermodel(objectProblem, homogeneousEvidence, dimensionBudget)
```

These are interface sketches, not names of functions already present in Forge. The last request must not receive authority to close an arbitrary Lean goal.

10.3 A proof-producing source bridge

For an expression e over an algebra A , reification should return an object polynomial p , an atom environment $\rho : X \rightarrow A$, and a Lean proof

$$\text{eval}_\rho(p) = e.$$

For every relation, it must return the analogous equation together with the actual local proof that the source relation is zero. The atom map is indexed by elaborated expressions in the exact context, not by display names.

The simplest supported grammar is literals, atoms, addition, subtraction, negation, multiplication, natural powers, and a certified central rational scalar action. For a nonrecognized subexpression, two safe choices exist: retain it as an opaque atom of the same algebra, or decline the problem. An opaque inverse is not supplied an inverse law automatically. For the push-through example, the source must prove or explicitly supply the two inverse relations; the worker never invents them from notation.

The bridge should establish its evaluation equations recursively. Atoms and constants are immediate; the add and multiply cases use the corresponding evaluation lemmas; a power follows by natural induction or a proved evaluation homomorphism. Star uses the declared involution agreement for every atom. This turns the search’s word identity into a theorem about the original expression without trusting serialization or pretty-printing.

Mathlib already provides `FreeAlgebra`, its universal lifting interface, and an equivalence with a monoid algebra on the free monoid [9]. These are useful semantic foundations. For executable checking, a canonical array/list representation is likely more convenient than reducing a quotient implementation directly. Prove its evaluation correspondence to the existing semantic object once; do not formalize a second abstract free-algebra theory merely to run sparse arithmetic.

Typed rectangular products. The delivered word language has one object: all atoms multiply in the same algebra. Rectangular matrix products need typed words with a source and target index type, and concatenation only when the middle types match. That extension can use paths in a finite typed graph, but it is not part of the current implementation. A preliminary adapter should reject mixed shapes rather than pad matrices or forget indices. A trace rotation for rectangular factors also moves between different matrix spaces; it needs the actual typed trace theorem, not a same-size word permutation.

10.4 Two reconstruction stages, with measurable gates

Stage one: theorem-producing replay. Decode a small receipt, reconstruct each word as a source expression, and ask existing normalization to prove the advertised free-algebra identity. For integer-coefficient identities, `noncomm_ring` is an immediate baseline reconstructor. Prove the zero contribution of each relation by congruence, multiplication by zero, and the supplied hypothesis. Sum the resulting proofs. This route avoids formalizing the search or a Gröbner algorithm, while allowing real kernel-checked examples early.

Rational coefficients need explicit scalar-action normalization. A tempting shortcut is multiplying through by a common denominator. That also creates a cancellation obligation: cancellation of a positive integer scalar is not valid in every ring. On a $\mathbb{Q}$ -algebra the scalar is invertible; the bridge

must use that structure explicitly. On an integer-only ring receipt, retain integer coefficients and use the simpler reconstruction instead.

Stage two: reflected replay. When proof terms from expanded identities become the bottleneck, define a computational checker over canonical word polynomials and prove

$$\text{checkEq}(p, r, c) = \text{true} \implies (\forall j, \text{eval}(r_j) = 0) \Rightarrow \text{eval}(p) = 0.$$

Analogous theorems interpret operator and trace receipts. The implementation must connect checked object evaluation to source reification before assigning a metavariable. A proved checker theorem still does not justify a mistranslated source goal.

The supplied `lean/CertificatePrinciples.lean` contains six small targets for this path: a sandwich-zero lemma, a finite-sum certificate principle, a commuted square, the resolvent and push-through identities, and positivity under matrix congruence. It does not implement the computational checker, certify all 119 JSON objects, or expose a new tactic. The compiler gate records `NOT_RUN` because no Lean executable was available.

10.5 Concrete matrix library dependencies

The matrix reconstruction should target `Matrix.PosSemidef` directly, avoiding an ambiguous generic `0 <= M`. Mathlib documents the needed operations: `Matrix.PosSemidef.conjTranspose_mul_mul_same`, `Matrix.PosSemidef.add`, `Matrix.PosSemidef.smul`,

`Matrix.posSemidef_sum`, and `Matrix.PosSemidef.trace_nonneg` [10]. The Loewner-order API can be introduced in the surface layer after confirming its scoped instance; the certificate core need not depend on that notation.

For a sandwich receipt, instantiate the congruence theorem at the decoded Q and the actual proof of G 's positive semidefiniteness. Use exact rational sign proofs for $d_k \geq 0$, scalar multiplication, and finite sums. The final word identity replaces the positive sum by the source target. For trace receipts, apply trace linearity and a proved cyclicity identity to the decoded commutators; the resulting theorem is scalar-valued and stays scalar-valued when returned to the host.

10.6 Concrete optional search backends

Backend	Useful existing facility	Adapter obligation and limitation
Python standard library	Exact fractions, dictionaries, deterministic worklists	The only executed producer/replayer substrate in this package. Suitable as a reference, not a production performance claim.
FLINT / python-flint	Rational matrix row reduction and solving, including <code>fmpq_mat.rref()</code> and <code>solve()</code> [12]	Convert exact numerators/denominators, retain column labels, and re-expand returned solutions. Backend matrix conventions do not define receipt semantics.

Backend	Useful existing facility	Adapter obligation and limitation
NCAlgebra	NCPoly, NCGBX, and bounded NCPolyGroebner; symbolic/numeric SDP support through NCSDP [13]	Useful untrusted Wolfram Language search. Export original two-sided context multipliers, not just a zero remainder. Requires a separate installed host; not invoked here.
Ncpol2sdpa	Sparse moment/SDP relaxations for noncommutative polynomial optimization [14, 15]	Use for candidate Gram data or supports. A moment relaxation bound or floating solver verdict is not an accepted certificate. The appropriate primal/dual conversion must be implemented and checked.
Mathlib	Free-algebra semantics, existing normalization, matrix positivity lemmas [9, 8, 10]	Proof reconstruction and final authority. Neither an installed compatible package set nor a successful build is claimed by this dependency table.

For mixed-degree SDP search, let W contain the chosen words, introduce a symmetric Gram matrix for each positive premise, and impose coefficient equalities modulo the equality span. A numerical solve suggests supports or approximate matrices. Reconstruct rational entries by solving the exact affine coefficient equations on that support; then test each rational matrix by exact LDL . If repair fails, reject the proposal. Merely rounding entries and checking that a numerical eigenvalue is “almost nonnegative” is not a repair algorithm.

This is a proposed backend path. It extends the coefficient language in the same way as the executed finite-ray search, but does not inherit that routine’s finite support completeness claim. Exact rational repair on a chosen numerical support can miss feasible solutions. It must remain an untrusted candidate producer with a distinct status.

For NCAlgebra, the current documentation recommends the pure-Wolfram NCGBX route rather than the old C++ component [13]. Even a successful noncommutative completion is only a discovery mechanism here; a nonterminating or iteration-limited completion yields UNKNOWN. The homogeneous degree procedure supplies a simpler exact alternative when its entrance conditions hold.

10.7 A routing algorithm the planner can implement

```

route(problem):
    reject unsupported scalar domain, source operations, or shape erasure
    try cheap existing normalization and proved local rewrites first
    if algebra equality:
        if all relations homogeneous:
            run context search at target degree
            replay identity if found
            otherwise construct and replay quotient matrices within budget
        else:
            run bounded context levels; all failures stay UNKNOWN
    if operator positivity:
        discharge or expose missing source self-adjointness obligations
        if free homogeneous even-degree target and no needed constraints:

```

```

    try unique-Gram exact LDL
    try relation-derived rays plus explicit source-supplied rays
    optionally request an external candidate Gram certificate
  if trace positivity:
    keep the trace wrapper
    use cyclic residual search in the implemented fragment
  export only a proof of the original proposition, or a scoped diagnostic

```

The homogeneous Gram fast path can safely ignore additional assumptions when it proves the target unconditionally, but a failure on that weaker unconstrained problem must not refute the constrained goal. Similarly, equality receipts may use a subset of available relations for positive proof; algebra countermodels must satisfy every relation included in the claimed universal object problem. The original source context may be richer still, which is why a source-level negative verdict needs a stronger translation gate.

A useful exchange with Leant-style typed synthesis is to request candidate operator expressions q from an exact local inventory, then pass those expressions to the positive-ray search. A useful exchange in the other direction is to return an equality that simplifies the behavioral specification of a candidate. The first implementation need not support unrestricted higher-rank synthesis: the executed involution dictionary already exercises the proposal/verification split with fully explicit algorithms.

10.8 Proof size and cache discipline specific to this worker

Equality elimination provenance can become large even when the final target has a short proof. Prefer a shared directed acyclic graph for repeated context multiplications, followed by dead-term removal and exact coefficient collection. The current JSON format expands the final list and does not implement DAG compression. The slicing experiment shows a different benefit: a three-term receipt can be found without building an elimination basis of rank 1,345.

Memoizing the equality remainder basis is useful across repeated demands from one presentation. Its cache key must include the exact ordered atom meanings, coefficient domain, complete relation list, context bound, and involution when used for positivity. A cache may provide *search guidance* after a looser match; it may not provide an accepted proof. The graph dictionary can be cached separately as proposals, but a cached square still needs its source typing and current receipt replay.

The supplied checker imposes finite input sizes and product counts, rejects booleans as integers, requires canonical rational encodings, rejects duplicate JSON keys, and fails closed on malformed evidence. These are useful research checks, not a claim of a hardened parser for hostile internet workloads. A production process boundary also needs byte limits before JSON decoding, cooperative cancellation, wall-clock and memory limits, and failure isolation.

11 An incremental implementation plan with acceptance tests

11.1 Gate A: one real source-level equality

Implement rational-word reification and theorem-producing equality replay. Accept a Lean goal only after producing a proof of its exact source expression. Run the resolvent and push-through examples in a fresh Lean process, record compiler/toolchain and Mathlib revisions, and inspect the actual

axiom inventories. Include changed-target, changed-multiplication-instance, and characteristic-two controls. The delivered prototype is not a substitute for this gate.

11.2 Gate B: reusable equality search

Add exact context enumeration and incidence slicing. Compare the two paths on identical Lean goals with the same reconstruction and resource limits. Measure search time, reconstruction time, kernel time, peak memory, and proof size separately. A small Python elimination time is not a prediction of Lean elaboration time. Add multiple demands sharing a relation basis, but preserve source-level context identities between demands.

11.3 Gate C: operator and trace proof reconstruction

Port the star correspondence and sandwich/trace interpretation theorems. Require the self-adjoint projection and positive congruence examples to close source goals. Then replay the homogeneous degree-six result and the two-stage CHSH derivation. Reject the trace/operator confusion example as an operator proof. No external SDP integration should precede these positive and negative controls: the hard boundary is reconstruction, not sophisticated discovery.

11.4 Gate D: exact-fragment diagnostics

The first production countermodel feature can be a diagnostic for an explicitly reified universal-algebra query. Verify whole relation matrices and the target vector in Lean or a separately verified matrix checker. Only then consider connecting it to source-goal refutation, with fixed dimensions and all local assumptions accounted for. A proved impossibility result for the object language must never become a proof of arbitrary Lean non-inhabitation.

11.5 Gate E: broader search and a fair comparative corpus

Only after the earlier gates should the project integrate a numerical SDP or noncommutative completion backend. Construct a held-out corpus of symbolic matrix, endomorphism, projection, and trace goals. Include failed goals and true goals outside the worker's fragments, not just generated certificates. Compare existing tactics, a noncommunicating portfolio, and the integrated worker. Report solved original goals under a fixed budget, not counts of internal algebra subgoals or generated receipt terms. The intended milestone is a measured increase in kernel-checked source goals, not a larger Python certificate corpus.

12 Executed experiments and their limits

12.1 Environment and independent replay

The recorded run used Python 3.13.5 on Linux x86-64 and seed 2026091503. Every experiment, test, and replay command used `python -S`, so site packages were disabled. No SymPy, NumPy, SDP solver, FLINT, Wolfram kernel, Leant, or Djex executable was needed for the run. The raw environment string, per-family timings, and outcomes are stored in `results/experiments.json`. Timing values are single-process observations on this environment, not statistically stable performance estimates.

The standalone replay script installs an import guard that rejects imports of `ncforge.algebra`, `ncforge.search`, and `experiments`. It then loads the stored external problems and certificates, validates their shapes, and recomputes every identity or matrix witness using only `ncforge.checker`. It replayed all 119 stored objects without failure. This demonstrates that the stored objects are sufficient for the independent Python checker, not that the checker is formally verified.

12.2 Three exact equality oracles

The generated equality corpus contains three families of thirty queries each. Each family interleaves targets planted in the degree-bounded relation span with arbitrary sparse targets. The expected verdict comes from a mathematically different oracle, not from the producer’s zero-remainder test.

Commuting generators.

The relations make two generators commute. The oracle groups coefficients by ordinary exponent counts; membership is equivalent to the resulting commutative polynomial being zero. It does not run noncommutative elimination.

Monomial relations.

The relations are $x^2 = 0$ and $xyx = 0$. A word lies in their monomial ideal exactly when it contains one of these forbidden words as a contiguous substring. The oracle uses that substring predicate independently on every target monomial.

Exterior relations.

Three generators square to zero and pairwise anticommute. A repeated generator makes a word zero; otherwise sorting its letters produces a sign determined by the number of inversions. The oracle collects these signed normal forms rather than computing a rational relation basis.

All targets have degree at most three. Each family produced fifteen positive equality receipts and fifteen checked rational matrix countermodels, with no oracle disagreements. Maximum witness dimensions were ten, ten, and eight, respectively. The counts are outcomes on ninety selected object-language queries, not evidence that ninety previously unsolved Lean goals now close.

Family	Queries	Equality receipts	Matrix witnesses	Disagreements
Commuting	30	15	15	0
Monomial	30	15	15	0
Exterior	30	15	15	0

Table 5: Exact equality outcomes. Negative objects satisfy all relation matrices, not just sampled relation evaluations.

12.3 Forty homogeneous Gram queries

The homogeneous-square experiment uses four-by-four rational Gram matrices for two-letter words of length two, hence degree-four free polynomials. Twenty matrices are generated as $A^\top A$; twenty are independently generated symmetric matrices. The oracle checks every principal minor by a separate dense determinant routine. The producer instead uses pivoted *LDL*.

The outcome is twenty accepted square receipts and twenty cases outside the homogeneous square cone, with no disagreement against the exact principal-minor oracle. All forty queries are reproducible

from the recorded seed. The twenty nonreceipt inputs and their Gram data are additionally stored in `results/negative-gram-queries.json`. They are not included among the 119 replayable positive/countermodel objects, and no matrix-nonpositivity witness is claimed for them.

12.4 Nine worked obligations

The worked corpus contributes nine receipts, with distinct roles:

Name	Mechanism exercised
Square-zero sum	Three homogeneous relations combined at degree two.
Resolvent identity	Inhomogeneous two-sided relation contexts.
Push-through	Degree-five proof of a degree-two target; lower-bound controls.
Projection positivity	A square plus an equality residual.
Positive congruence	A nontrivial positive premise inside a sandwich.
Degree-six Gram	Automatic factor recovery without a supplied dictionary.
Trace, not operator	Cyclic residuals with a concrete negative operator test.
CHSH operator bound	Automatically generated six-ray dictionary, rational weights, and equality reconstruction.
CHSH self-adjointness	A separate source-shape obligation for the squared form.

The two CHSH receipts are two obligations in one derivation, not independent end-to-end theorems for benchmark purposes. Altogether the stored evidence count is $90 + 20 + 9 = 119$: forty-five equality countermodels, forty-five generated equality receipts, twenty homogeneous-square receipts, and nine worked receipts. This is a storage count, not a combined success rate over different tasks.

12.5 An ablation with the same producer and target

For $m = 3, 4, 5, 6$, give every pair of atoms a commutation relation and ask for $x_0^3 x_1 - x_1 x_0^3 = 0$ at context bound four. Run the same exact producer with component slicing disabled and enabled. Both routes return three-term receipts. The complete results are in `results/slicing.csv`.

Atoms	Assembled columns	Sliced columns	Full rank	Full time (ms)	Sliced (ms)
3	102	3	86	2.919	0.144
4	342	3	271	24.139	0.347
5	860	3	655	139.391	0.759
6	1,815	3	1,345	572.457	1.602

Table 6: Single-run Python ablation. Both paths assemble all context columns; only the elimination component is reduced. Sliced rank is three in every row.

The strongest stable observation is structural: at six atoms, only three of 1,815 assembled columns need elimination, while the unsliced basis has rank 1,345. The elapsed time changes from about 572 milliseconds to about 1.6 milliseconds in this recorded run. Those are not Lean timings, and the ratio on one deliberately structured family is not a general speedup estimate. In particular, this family is chosen to make unused commutation components visible; a densely connected target could retain almost every column.

12.6 Tests and mutation audit

The unit suite reports **57 passing test methods**. One method includes all 24 atom-renaming subtests of the CHSH construction; these are not silently added to the method count. The suite covers exact fraction arithmetic, star reversal and atom mapping, zero and singular Gram branches, elimination provenance, slicing, unsupported inhomogeneous quotient construction, and resource refusal. Its semantic controls include characteristic-domain metadata refusal, unsupported fixed-dimension metadata, positivity-premise deletion, and the trace/operator distinction.

The certificate mutation audit records **195 attempted invalid objects, with zero incorrectly accepted**. Its deterministic attacks replace the original target while keeping a receipt, erase a nonzero ideal contribution, negate a square weight, or zero a separating vector. These are deliberately falsifying mutations, not a claim that arbitrary modifications to a valid receipt must fail: some modifications, such as adding a zero summand, preserve correctness. Detailed per-object outcomes are retained in `results/mutations.json`.

Additional unit attacks cover swapped left/right context placement, dropped or replaced relations, float or Boolean coefficients, zero denominators, unreduced fractions, duplicate monomials, malformed involutions, out-of-range letters, altered certificate interpretation, duplicate JSON keys, and a relation that vanishes only on the witness vector. The last control is particularly important for the graded countermodel construction.

Mutation rejection is useful evidence against concrete implementation errors. It is not a proof of checker correctness, and zero accepted mutations does not measure the probability of an undiscovered bug.

12.7 What was not executed

There was no Lean compiler or Lake executable in the authoring environment. The explicit compiler gate therefore returned `NOT_RUN`; its JSON record is included. None of the supplied Lean files is represented as elaborated, and no source-to-object correspondence was tested. No comparison with `grind`, `noncomm_ring`, Aesop, or an integrated Forge prototype was run. No external linear-algebra, completion, or SDP backend was invoked.

The report’s mathematical proofs establish the stated abstract contracts by ordinary mathematical reasoning. They have not themselves been formalized. The executed evidence establishes the behavior of the delivered Python programs on the recorded corpus. Conflating those two statements would overstate the result, even when every tested certificate is correct.

13 Reproducing and extending the package

13.1 Files that matter

Path	Purpose
<code>article/forge-noncommutative.tex</code>	Main LaTeX source, with five section files, the recorded ablation table, and a conventional bibliography.
<code>article/forge-noncommutative.pdf</code>	Compiled article.

Path	Purpose
prototype/ncforge/algebra.py	Producer-side exact word arithmetic and provenance-carrying elimination.
prototype/ncforge/search.py	Equality, graded countermodel, Gram, finite-ray, cyclic, and graph-dictionary producers.
prototype/ncforge/checker.py	Separate canonical decoding, identity replay, and whole-matrix countermodel replay.
prototype/experiments.py	Seeded oracle corpus, worked obligations, and the slicing ablation.
prototype/replay.py	Stored-evidence replay with producer imports blocked.
prototype/tests/	Unit and semantic-boundary tests.
prototype/mutation_audit.py	Per-object falsifying mutation checks.
results/	Original run data, certificates, nonreceipt inputs, tests, mutations, replay record, initial failure, and Lean status.
lean/CertificatePrinciples.lean	Six uncompiled reconstruction targets.
tools/check_lean.py	Compiler gate requiring an existing built Mathlib project.
docs/	Missing tools are not successes.
	Source audit, schema contract, limitations, and failure note.

13.2 Reproduction commands

From the package root:

```
cd prototype
python -S -m unittest discover -s tests -v
python -S replay.py ../results/certificates.json
python -S experiments.py --output ../reproduced-results
python -S mutation_audit.py
```

The experiment command writes new output separately by default, so the original recorded timings are not overwritten. The mutation script reads the recorded corpus and regenerates its mutation report under `results/`; the README makes that local-file behavior explicit. All commands are local and make no network calls.

To compile the article, run `tools/build_article.sh`, or run pdfLaTeX three times in `article/`. To test the uncompiled Lean targets in an already built compatible Mathlib checkout:

```
python tools/check_lean.py --project /path/to/mathlib-project
```

A successful invocation would validate those six targets only. It would still not validate the absent reifier, reflected checker, worker integration, or full JSON corpus. A fresh, pinned project should preserve and report its actual compiler output and axiom inventory.

13.3 A direct Python use example

```
from ncforge.algebra import var, mul, add, power, problem
from ncforge.search import equality_search
from ncforge.checker import verify
```

```

x, y = var(0), var(1)
relation = add(mul(x, y), mul(y, x), -1)
target = add(mul(power(x, 2), y), mul(y, power(x, 2)), -1)
original = problem(target, [relation], 2)
answer = equality_search(target, [relation], 2, D=3)
if answer["status"] == "certificate":
    result = verify(original, answer["certificate"])
    assert result["accepted"]
else:
    print(answer["reason"]) # a bounded search failure, not a false theorem

```

The public research API returns Python dictionaries rather than enforcing an opaque capability type. A production Forge adapter should wrap successful *Lean* reconstruction in the host’s already-designed proof-receipt boundary. The Python `accepted` field has no authority to assign a Lean metavariable.

14 Conclusion

The main recommendation is to add one coherent, order-preserving algebra worker rather than another general search slogan. Two-sided word receipts cover relations inside arbitrary multiplication contexts. Homogeneous presentations admit a degree-complete positive/negative alternative with explicit finite rational matrices. Unique homogeneous word Gram matrices give an automatic Hermitian-square fragment beyond scalar quadratic arithmetic. Conditional operator inequalities require adjoint sandwiches, while trace inequalities permit explicitly reconstructed commutator residuals and must remain a separate output kind.

The implemented prototypes make those distinctions executable. They also show where a planner can profit from cooperation: an equality proof establishes the self-adjointness needed by an operator goal, equality-space reduction simplifies positive-weight search, and structural relations propose useful squares before any numerical backend is considered. The tested incidence slice is an exact search reduction with a small proof, not a heuristic declaration that ignored hypotheses are irrelevant.

The next decisive artifact is not a larger survey or more generated examples. It is a Lean source bridge and a small set of original goals closed by kernel-checked reconstruction. The current package supplies the algorithms, mathematical contracts, executable reference behavior, and rejection controls needed to make that implementation concrete.

References

- [1] V. Reshetnikov and contributors. *Forge: README and merged article provenance*, inspected 15 September 2026, commit `c98e47c5f804e92880fc1d0e378c1b95832b685c`. <https://github.com/VladimirReshetnikov/Forge/tree/c98e47c5f804e92880fc1d0e378c1b95832b685c>. Relevant files: `README.md`, `article/sections/02-baseline.tex`, and `article/sections/B-provenance.tex`.
- [2] Forge contributors. *Merged nonlinear-arithmetic section*, merged article, `article/sections/07-nonlinear.tex`, same pinned revision as [1]. <https://github.com/VladimirReshetnikov/Forge/blob/c98e47c5f804e92880fc1d0e378c1b95832b685c/article/sections/07-nonlinear.tex>.

- [3] Forge contributors. *Finite certificates for unbounded behaviour*, merged article, `article/sections/06-closure.tex`, same pinned revision as [1]. <https://github.com/VladimirReshetnikov/Forge/blob/c98e47c5f804e92880fc1d0e378c1b95832b685c/article/sections/06-closure.tex>.
- [4] Forge contributors. *Ideas from Leant and Djex*, `docs/IDEAS-FROM-LEANT-DJEX.md`, same pinned revision as [1]. <https://github.com/VladimirReshetnikov/Forge/blob/c98e47c5f804e92880fc1d0e378c1b95832b685c/docs/IDEAS-FROM-LEANT-DJEX.md>.
- [5] V. Reshetnikov and contributors. *Leant: a Djex-based synthesis REPL for Lean 4*, README inspected at commit `6bf05ad78c467989e68290f2d08bbbed40802d485`. <https://github.com/VladimirReshetnikov/Leant/tree/6bf05ad78c467989e68290f2d08bbbed40802d485>.
- [6] V. Reshetnikov and contributors. *Djex*, README inspected at commit `e8778f4ebd63e1f9b9b410fa4de8d14a8a04c9e5`. <https://github.com/VladimirReshetnikov/Djex/tree/e8778f4ebd63e1f9b9b410fa4de8d14a8a04c9e5>.
- [7] Lean developers. *The grind tactic*, Lean Language Reference, consulted 15 September 2026. <https://lean-lang.org/doc/reference/latest/The--grind--tactic/>.
- [8] Mathlib contributors. *Mathlib.Tactic.NoncommRing*, live API documentation, consulted 15 September 2026. https://leanprover-community.github.io/mathlib4_docs/Mathlib/Tactic/NoncommRing.html.
- [9] Mathlib contributors. *Mathlib.Algebra.FreeAlgebra*, live API documentation, consulted 15 September 2026. https://leanprover-community.github.io/mathlib4_docs/Mathlib/Algebra/FreeAlgebra.html.
- [10] Mathlib contributors. *Mathlib.LinearAlgebra.Matrix.PosDef*, live API documentation, consulted 15 September 2026. https://leanprover-community.github.io/mathlib4_docs/Mathlib/LinearAlgebra/Matrix/PosDef.html.
- [11] J. W. Helton. “Positive” noncommutative polynomials are sums of squares. *Annals of Mathematics* **156**(2) (2002), 675–694. <https://doi.org/10.2307/3597203>. Publisher abstract and bibliographic record: <https://annals.math.princeton.edu/2002/156-2/p09>.
- [12] python-flint developers. *fmpq_mat: matrices over rational numbers*, live documentation, consulted 15 September 2026. https://python-flint.readthedocs.io/en/latest/fmpq_mat.html.
- [13] NCAlgebra developers. *The NCAlgebra Suite*, current reference manual, particularly NCPolyGroebner, NCGBX, and NCSDP; consulted 15 September 2026. <https://ncalgebra.github.io/>.
- [14] Ncpol2sdpa developers. *Ncpol2sdpa documentation*, stable documentation, consulted 15 September 2026. <https://ncpol2sdpa.readthedocs.io/en/stable/>.
- [15] P. Wittek. Algorithm 950: Ncpol2sdpa—Sparse semidefinite programming relaxations for polynomial optimization problems of noncommuting variables. Preprint, arXiv:1308.6029 (2013). <https://arxiv.org/abs/1308.6029>.